

Specyfikacja Projektu Zaliczenowego (Programowanie w Języku Java)

Maksymilian Piwowar, Jakub Sewiło, Jakub Szlachetka

11.04.2026

1. Tytuł Projektu

ChatMM - Aplikacja Czat z Szyfrowaniem End-to-End

2. Opis i Cele Projektu

Projekt ma na celu stworzenie aplikacji czatu w czasie rzeczywistym, która zapewnia pełne szyfrowanie end-to-end wiadomości wymienianych między użytkownikami. Komunikacja odbywa się za pomocą protokołu WebSocket, a każda wiadomość jest szyfrowana po stronie klienta przed wysłaniem na serwer - serwer nigdy nie ma dostępu do treści wiadomości w postaci jawnej. Główne cele projektu:

- Bezpieczna rejestracja i logowanie użytkowników z wykorzystaniem tokenów JWT.
- Szyfrowanie wiadomości end-to-end z użyciem algorytmów AES i RSA.
- Obsługa wielu użytkowników w czasie rzeczywistym.
- Przechowywanie historii zaszyfrowanych wiadomości w bazie danych PostgreSQL.
- Intuicyjny graficzny interfejs użytkownika zbudowany w React (Next.js).

3. Wymagania Funkcjonalne

- Rejestracja konta użytkownika z weryfikacją danych (unikalny login, email, hasło).
- Logowanie z uwierzytelnianiem JWT (access token + refresh token).
- Generowanie pary kluczy RSA (publiczny/prywatny) po stronie klienta przy rejestracji.
- Publikacja klucza publicznego użytkownika na serwerze (przechowywany w bazie danych).

- Tworzenie prywatnych konwersacji (1:1) oraz grupowych pokoiów czatu.
- Szyfrowanie wiadomości kluczem AES, który jest szyfrowany kluczem RSA odbiorcy.
- Wysyłanie i odbieranie wiadomości w czasie rzeczywistym przez WebSocket.
- Przechowywanie historii wiadomości (w postaci zaszyfrowanej) w bazie danych.
- Wyszukiwanie użytkowników po nazwie w celu inicjowania konwersacji.
- Wyświetlanie statusu wiadomości: wysłana, dostarczona, odczytana.
- Możliwość usuwania konta i danych przez użytkownika.

4. Wymagania Niefunkcjonalne

- **Wydażność:** Opóźnienie dostarczenia wiadomości poniżej 200 ms w sieci lokalnej.
- **Skalowalność:** Obsługa minimum 100 jednoczesnych połączeń WebSocket.
- **Bezpieczeństwo:** Cała komunikacja przez HTTPS/WSS; hasła szyfrowane algorytmem bcrypt; szyfrowanie E2E uniemożliwia odczyt wiadomości po stronie serwera.
- **Użyteczność:** Responsywny interfejs.
- **Zgodność:** Aplikacja działa na przeglądarkach.
- **Niezawodność:** Automatyczne wznowianie połączenia WebSocket po utracie łączności.

5. Architektura Systemu

Typ architektury: **Klient-Serwer (REST + WebSocket)**.

Schemat:

- **Frontend:** Next.js (React) + TailwindCSS - renderowanie SSR(Server-Side Rendering)/CSR (Client-Side Rendering), zarządzanie stanem, obsługa WebSocket po stronie klienta, szyfrowanie/deszyfrowanie wiadomości w przeglądarce (Web Crypto API).
- **Backend:** Java (Spring Boot) - REST API, zarządzanie połączeniami WebSocket (STOMP), logika biznesowa, autoryzacja JWT.
- **Baza danych:** PostgreSQL - przechowywanie użytkowników, kluczy publicznych RSA, zaszyfrowanych wiadomości i metadanych konwersacji.
- **WebSocket:** STOMP over WebSocket (Spring WebSocket) - kanał komunikacji real-time.
- **Autoryzacja:** JWT (access token ważny 15 min + refresh token ważny 7 dni).

6. Zastosowane Algorytmy

- **Szyfrowanie asymetryczne RSA-2048:** Przy rejestracji użytkownika generowana jest para kluczy RSA. Klucz publiczny jest przesyłany na serwer i przechowywany w bazie danych. Klucz prywatny pozostaje wyłącznie po stronie klienta.
- **Szyfrowanie symetryczne AES-256-GCM:** Każdy chat jest szyfrowany losowo generowanym kluczem sesyjnym AES-256.
- **Algorytmy uwierzytelniania JWT:** Tokenizacja i walidacja JWT z podpisem HMAC-SHA256. Access token zawiera identyfikator użytkownika i rolę; refresh token służy do odnawiania sesji bez ponownego logowania.
- **Haszowanie haseł:** Hasła użytkowników przechowywane w bazie danych są poddawane hashowaniu.
- **Zarządzanie sesją WebSocket:** Przy nawiązaniu połączenia WebSocket token JWT jest weryfikowany.

7. Struktura Klas

Model (JPA Entities)

- **User** - id, username, email, passwordHash, publicKeyRsa, privateKeyRsaHash, privateKeyRsaSalt, registryDate, lastActive.
- **Conversation** - id, name, listOfParticipants, dateofCreation, groupKeyAESHash.
- **Message** - id, conversationId, senderId, encryptedContent, iv, timestamp, status.
- **RefreshToken** - id, userId, token, expiryDate.

Serwisy

- **UserService** - rejestracja, wyszukiwanie użytkowników, zarządzanie kluczami publicznymi.
- **AuthService** - logowanie, generowanie i walidacja tokenów JWT, obsługa refresh tokenów.
- **ConversationService** - tworzenie i zarządzanie konwersacjami prywatnymi.
- **MessageService** - zapis zaszyfrowanych wiadomości, pobieranie historii, obsługa statusów.
- **WebSocketService** - rozsyłanie wiadomości do subskrybentów.

Kontrolery

- AuthController - POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh.
- UserController - GET /api/users/search, GET /api/users/{id}/publicKey.“
- ConversationController - GET/POST /api/conversations, GET /api/conversations/{id}/messages.
- MessageController - obsługa WebSocket: /app/chat.send, /topic/conversation.{id}.

Dodatkowe komponenty

- JwtFilter - filtr Spring Security weryfikujący token JWT przy każdym żądaniu HTTP.
- WebSocketAuthInterceptor - weryfikacja JWT przy handshake WebSocket.
- DTO - MessageDTO, UserDTO, ConversationDTO - obiekty wymiany danych.
- Repository - interfejsy Spring Data JPA: UserRepository, MessageRepository, ConversationRepository.

8. Wykorzystane Technologie i Biblioteki

Frontend:

Next.js (React), TailwindCSS, ShadCN/UI, React Query (TanStack Query), SockJS + STOMP.js (WebSocket), Web Crypto API (szyfrowanie E2E w przeglądarce).

Backend:

Spring Boot 3, Spring Security, Spring WebSocket (STOMP), Spring Data JPA, Hibernate, JWT (biblioteka JWT).

Baza Danych:

PostgreSQL - relacyjna baza danych przechowująca użytkowników, zaszyfrowane wiadomości, klucze i metadane konwersacji.

DevOps:

Docker.

9. API i Modele Danych

Przykładowe Endpointy REST:

POST /api/auth/register

POST /api/auth/login

POST /api/auth/refresh

GET /api/users/search?q={query}

GET /api/users/{id}/publicKey

GET /api/conversations

POST /api/conversations

GET /api/conversations/{id}/messages

Przykładowe Endpointy WebSocket (STOMP):

SUBSCRIBE /topic/conversation.{conversationId}

SEND /app/chat.send

Model Danych - Wiadomość:

```
{
  "id": "uuid",
  "conversationId": "uuid",
  "senderId": "uuid",
  "encryptedContent": "base64_string",
  "encryptedAesKey": "base64_string",
  "iv": "base64_string",
  "timestamp": "ISO8601",
  "status": "SENT"
}
```

Model Danych - Użytkownik:

```
{
  "id": "uuid",
  "username": "string",
  "email": "string",
  "passwordHash": "string",
  "publicKeyRsa": "base64_string",
  "privateKeyRsaHash": "string",
  "privateKeyRsaSalt": "string",
  "registryDate": "ISO8601",
  "lastActive": "ISO8601"
}
```

10. Interfejs Użytkownika

Główne widoki:

- **Ekran logowania i rejestracji:** Formularze z walidacją danych, widoczny wskaźnik siły hasła, komunikaty o błędach.
- **Lista konwersacji:** Panel boczny z listą aktywnych czatów, podgląd ostatniej wiadomości, licznik nieprzeczytanych wiadomości.
- **Okno czatu:** Widok wiadomości w czasie rzeczywistym, pole tekstowe z przyciskiem wysyłania, statusy dostarczenia.
- **Wyszukiwanie użytkowników:** Pasek wyszukiwania do odnajdowania innych użytkowników i inicjowania nowych konwersacji.
- **Ustawienia konta:** Zmiana hasła, wylogowanie ze wszystkich urządzeń, usunięcie konta.
- **Panel administracyjny:** Zarządzanie użytkownikami, podgląd aktywnych sesji, blokowanie kont.

Elementy UX/UI:

- Responsywność - układ dwupanelowy na desktopie, pełnoekranowy widok czatu na urządzeniach mobilnych.
- Tryb ciemny i jasny dostępny z poziomu ustawień.
- Widoczny wskaźnik szyfrowania E2E przy każdej konwersacji (ikona kłódki + etykieta).
- Animacje ładowania wiadomości i płynne przejścia między ekranami.

Technologie UI:

Next.js, TailwindCSS, ShadCN/UI, SockJS + STOMP.js.

11. Etapy Realizacji / Harmonogram

Faza	Opis	Czas
Planowanie	Określenie wymagań, projekt bazy danych, wybór bibliotek kryptograficznych	1 tydzień
Backend	Budowa REST API, modele JPA, JWT, WebSocket	2 tygodnie
Frontend	Interfejs Next.js, Web Crypto API, integracja WebSocket	2 tygodnie
Integracja i testy	Testy E2E szyfrowania, testy jednostkowe i integracyjne	1 tydzień

Wdrożenie	Konfiguracja Docker, CI/CD, serwer produkcyjny	1 tydzień
-----------	--	-----------

12. Testowanie

- Testy jednostkowe backendu - JUnit 5 + Mockito (serwisy, logika JWT, szyfrowanie).
- Testy integracyjne REST API - Spring Boot Test + Testcontainers (PostgreSQL w kontenerze).
- Testy WebSocket - symulacja połączeń wielu klientów, weryfikacja dostarczenia wiadomości.
- Testy E2E szyfrowania - weryfikacja, że serwer nie otrzymuje wiadomości w postaci jawnej.
- Testy interfejsu użytkownika - Cypress (scenariusze logowania, wysyłania wiadomości).
- Testy obciążeniowe - symulacja 100 jednoczesnych użytkowników na WebSocket.

13. Ryzyka i Środki Zaradcze

Ryzyko	Wpływ	Zarządzanie
Błędy implementacji kryptografii (nieprawidłowa obsługa IV, padding)	Wysoki	Stosowanie sprawdzonych bibliotek (Web Crypto API); przeglądy kodu
Utrata klucza prywatnego RSA po stronie klienta	Wysoki	Dokumentacja z ostrzeżeniem; opcjonalny mechanizm kopii zapasowej klucza chronionej hasłem
Opóźnienia WebSocket przy dużym ruchu	Średni	Buforowanie wiadomości; optymalizacja rozgłaszania w Spring WebSocket
Błędy bezpieczeństwa JWT (wyciek refresh tokenu)	Wysoki	Przechowywanie refresh tokenu w HttpOnly cookie; rotacja tokenów przy każdym użyciu
Opóźnienia harmonogramu	Średni	Praca iteracyjna; priorytetyzacja funkcji kluczowych

14. Kryteria Sukcesu

- Wszystkie funkcje działają zgodnie ze specyfikacją (czat real-time, szyfrowanie E2E, JWT).
- Pokrycie testami backendu powyżej 80%.
- Weryfikacja, że serwer nie przechowuje ani nie przetwarza wiadomości w postaci jawnej.
- Czas dostarczenia wiadomości poniżej 200 ms w środowisku testowym.
- Brak błędów krytycznych związanych z bezpieczeństwem.

15. Szacunkowe Nakłady Pracy (Workload Estimation)

Całkowity szacowany czas pracy: **165 godzin roboczych**.

Moduł / Zadanie	Opis	Szac. czas (h)
Analiza i projektowanie	Wymagania, model bazy danych, dobór bibliotek kryptograficznych	12
Backend API (REST + JWT)	Modele JPA, logika biznesowa, autoryzacja	35
Moduł kryptograficzny	Implementacja RSA/AES, integracja Web Crypto API	20
WebSocket (real-time)	STOMP, obsługa połączeń	15
Frontend UI (Next.js)	Komponenty, integracja API i WebSocket, szyfrowanie w przeglądarce	30
Uwierzytelnianie i sesje	Login, rejestracja, JWT refresh, HttpOnly cookies	10
Testy (unit + E2E)	Pokrycie testami kluczowych ścieżek i modułu kryptograficznego	15
Docker	Konfiguracja środowiska	10
Dokumentacja techniczna	README, opis API, instrukcja uruchomienia	8

Bufor czasowy / błędy	Debugowanie, przewidziane blemy	nie- pro-	10
-----------------------	---------------------------------------	--------------	----